

Deploy de Evolution API en Oracle Cloud Free Tier

Objetivo: levantar un servidor **Evolution API** siempre-encendido y gratis (Oracle Always Free ARM), con **HTTPS vía Cloudflare Tunnel**, listo para conectar desde STApp (Configuración → WhatsApp → Evolution).

Always-on es obligatorio: si el proceso se duerme/reinicia, se cae la sesión de WhatsApp. Oracle ARM no se duerme por idle → sirve.

Arquitectura



Cloudflare Tunnel hace una conexión **saliente** desde la VM; no exponés IP ni abrís puertos de entrada. Te da un hostname HTTPS estable que Vercel puede alcanzar.

Requisitos previos

- Cuenta **Oracle Cloud** (free tier; pide tarjeta para verificar, no cobra en Always Free).
- Un **dominio en Cloudflare** (puede ser uno barato; necesitás el dominio en CF para el Tunnel).
- Tu **API key secreta** para Evolution (generalá larga, ej. `openssl rand -hex 32`).

Parte 1 — Provisionar la VM ARM (Always Free)

1. Oracle Console → **Compute** → **Instances** → **Create Instance**.
2. **Image & shape**:
 - Image: **Canonical Ubuntu 22.04**.
 - Shape: **Ampere (ARM)** → **VM.Standard.A1.Flex**. Asigná **2 OCPU / 12 GB RAM** (entra holgado en Always Free: el límite es 4 OCPU / 24 GB total).
3. **SSH keys**: subí tu clave pública (o que Oracle genere el par y bajá la privada).
4. Create.

⚠ Gotcha conocido: las ARM Always Free suelen tirar **"Out of capacity"** según la región. Si pasa: probá otra **Availability Domain**, otra región, o reintentá en horario distinto. Es lo más molesto del free tier de Oracle.

Anotá la **IP pública** de la instancia.

Parte 2 — Acceso e instalación de Docker

SSH a la VM:

```
ssh ubuntu@TU_IP_PUBLICA
```

Instalá Docker + compose plugin:

```
sudo apt-get update && sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(.
/etc/os-release && echo $VERSION_CODENAME) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update && sudo apt-get install -y docker-ce docker-ce-cli
containerd.io docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker $USER && newgrp docker
docker --version
```

No hace falta tocar el firewall de Oracle ni iptables: el Tunnel sale por conexión saliente. (Por eso este camino evita el doble-firewall de Oracle.)

Parte 3 — docker-compose de Evolution + Postgres + Redis

Creá el proyecto:

```
mkdir -p ~/evolution && cd ~/evolution
```

~/evolution/docker-compose.yml:

```
services:
  evolution-api:
    image: atendai/evolution-api:v2.1.1 # fijá versión; cruzá con el repo si
    cambió
    restart: always
    depends_on:
      - postgres
      - redis
    environment:
```

```

- SERVER_URL=https://evo.tudominio.com      # tu hostname del Tunnel
(Parte 4)
- AUTHENTICATION_API_KEY=TU_API_KEY_SECRETA  # openssl rand -hex 32
- DATABASE_ENABLED=true
- DATABASE_PROVIDER=postgresql
- DATABASE_CONNECTION_URI=postgresql://evo:evopass@postgres:5432/evolution?
schema=public
- DATABASE_SAVE_DATA_INSTANCE=true
- DATABASE_SAVE_DATA_NEW_MESSAGE=false
- CACHE_REDIS_ENABLED=true
- CACHE_REDIS_URI=redis://redis:6379/0
- CACHE_REDIS_PREFIX_KEY=evolution
- CACHE_LOCAL_ENABLED=false
ports:
- "127.0.0.1:8080:8080"  # solo loopback; el Tunnel lo expone
volumes:
- evolution_instances:/evolution/instances

postgres:
  image: postgres:16-alpine
  restart: always
  environment:
    - POSTGRES_USER=evo
    - POSTGRES_PASSWORD=evopass
    - POSTGRES_DB=evolution
  volumes:
    - postgres_data:/var/lib/postgresql/data

redis:
  image: redis:7-alpine
  restart: always
  volumes:
    - redis_data:/data

volumes:
  evolution_instances:
  postgres_data:
  redis_data:

```

Levantá:

```

docker compose up -d
docker compose logs -f evolution-api  # esperá a que arranque limpio, Ctrl+C para salir

```

Smoke local (dentro de la VM):

```
curl -s http://127.0.0.1:8080/instance/fetchInstances \  
-H "apikey: TU_API_KEY_SECRET"
```

✅ Debe responder JSON (lista vacía [] al principio), no error de auth.

Las variables de entorno de Evolution **cambian entre versiones**. Si algo no arranca, cruzá con el `.env.example` del tag exacto que pusiste en `image`: (repo: github.com/evolution-foundation/evolution-api). Cambiá `evopass` por una contraseña real.

Parte 4 — HTTPS con Cloudflare Tunnel

1. Instalá cloudflared en la VM (ARM):

```
curl -L   
https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-  
linux-arm64 -o cloudflared  
sudo install -m 755 cloudflared /usr/local/bin/cloudflared  
cloudflared --version
```

2. Autenticá y creá el túnel:

```
cloudflared tunnel login      # abre URL, autorizás tu dominio en CF  
cloudflared tunnel create evolution
```

Anotá el **Tunnel ID** que imprime.

3. Config del túnel — `~/.cloudflared/config.yml`:

```
tunnel: TU_TUNNEL_ID  
credentials-file: /home/ubuntu/.cloudflared/TU_TUNNEL_ID.json  
  
ingress:  
- hostname: evo.tudominio.com  
  service: http://127.0.0.1:8080  
- service: http_status:404
```

4. Ruteá el DNS y corré como servicio:

```
cloudflared tunnel route dns evolution evo.tudominio.com  
sudo cloudflared service install  
sudo systemctl enable --now cloudflared
```

5. Verificá desde **afuera** (tu laptop):

```
curl -s https://evo.tudominio.com/instance/fetchInstances -H "apikey:
TU_API_KEY_SECRETA"
```

✓ Mismo JSON que el smoke local, pero ahora por HTTPS público. Eso es lo que STApp necesita.

Asegurate que `SERVER_URL` en el compose == `https://evo.tudominio.com`. Si lo cambiaste recién, `docker compose up -d` de nuevo para que tome el valor.

Parte 5 — Conectar en STApp

En STApp → **Configuración** → **WhatsApp** → **Evolution**:

- **Base URL:** `https://evo.tudominio.com`
- **Instance name:** ej. `taller-centro` (lo crea STApp al guardar)
- **API key:** `TU_API_KEY_SECRETA`

Seguí desde la **Parte B** de `docs/whatsapp-evolution-pruebas.md` (QR pairing → toggle de canal → prueba de cambio de estado).

Seguridad y operación

- **API key:** tratala como secreto. Cualquiera con la key + baseUrl controla tu WhatsApp. No la commitees.
- **Postgres/Redis:** en el compose quedan en red interna de Docker, sin puertos publicados. No los expongas.
- **Backups:** el volumen `postgres_data` + `evolution_instances` tiene la sesión. Si los perdés, hay que re-escanear el QR. Snapshot ocasional del block volume de Oracle.
- **Actualizaciones:** fijá la versión de la imagen (`v2.1.1`), no `latest`. Para actualizar: cambiá el tag, `docker compose pull` && `docker compose up -d`, y verificá que la sesión sobreviva.
- **Riesgo de baneo:** Evolution es WhatsApp **no oficial**. Para volumen alto o uso comercial serio, evaluá migrar a **Meta Cloud API** (oficial, con plantillas aprobadas) — STApp ya soporta ese provider también.
- **Costo Oracle:** dentro de Always Free no se cobra. Igual ponete límite de presupuesto en Billing por si te salís del free.

Troubleshooting

Síntoma	Causa probable	Acción
<code>curl</code> local OK pero público falla	Tunnel/DNS no propagó	<code>systemctl status cloudflared</code> ; revisá <code>route dns</code> y el hostname
Auth error en <code>fetchInstances</code>	<code>apikey</code> no matchea	Confirmá <code>AUTHENTICATION_API_KEY</code> en el compose

Síntoma	Causa probable	Acción
Evolution no arranca	Env var inválida para esa versión	Cruzá con <code>.env.example</code> del tag; mirá <code>docker compose logs evolution-api</code>
QR se conecta y se cae	VM se reinició / volumen perdido	<code>restart: always</code> ya está; verificá que los volúmenes persistan
"Out of capacity" al crear VM	Falta cupo ARM en la región	Otra AD/región o reintentar